StackCraft – Card Stacking Survival Game

Project Documentation

Table of Contents

1. Introduction 5
1.1 Template Overview 5
1.2 Target Audience 5
1.3 Support and Contact Information 6
2. Getting Started 6
2.1 Installation & Setup 6
2.1.1 Prerequisites 6
2.1.2 Importing the Template 6
2.1.3 Initial Configuration 8
2.1.4 Important Folders 8
2.2 Scene Structure 8

2.2.1 Director Layer (Persistent Singletons) 9
2.2.2 Gameplay Scene Hierarchy 9
2.2.3 Manager Organization (GameManagers Children) 10
2.3 Game Persistence 10
2.3.1 The GameData Structure 11
2.3.2 Saving and Loading Flow 11
2.3.3 Scene Travel 12
3. Core Game Loop: The Day Cycle 12
3.1 Overview of Time 12
3.2 Phase 1: Notification 13
3.3 Phase 2: Resource Management (Feeding & Selling) 13
3.3.1 Feeding Check (Survival) 13
3.3.2 Selling Check (Card Limit) 14
3.4 Phase 3: Encounters & Events 14
3.4.1 Encounter Selection 14
3.4.2 Encounter Execution 14
3.5 Phase 4: New Day 15
3.6 Game Over Condition 15
4. Game Systems Deep Dive 16
4.1 Card & Stack Management 16
4.1.1 Card Stacking Main Components 16
4.1.2 The Stacking Mechanic 16
4.2 Crafting & Industry 17
4.2.1 The Basics: How to Craft 17
4.2.2 Recipe Categories 18
4.2.3 Special Recipe Types 18
4.2.4 Ingredient Consumption 19
4.2.5 Recipe Discovery 19
4.3 Trading & Economy 19
4.3.1 Selling Cards (The Card Buyer) 19

4.3.2 Buying Packs (Pack Vendors) 20
4.3.3 Unlocking Vendors 20
4.3.4 Collection Tracker 20
4.4 Combat & Conflict 21
4.4.1 Initiating Combat 21
4.4.2 Combat Attributes (Stats) 21
4.4.3 The Combat Loop 22
4.4.4 Types & Advantages (Rock-Paper-Scissors) 22
4.4.5 Automated AI & Aggression 22
4.4.6 Ending Combat 22
4.5 Quest System 23
4.5.1 Quest Structure 23
4.5.2 Progression & Unlocks 23
4.5.3 Quest Types & Objectives 23
4.6 Encounter System 24
4.6.1 Overview 24
4.6.2 Core Architecture 24
4.6.3 Selection Logic 25
4.6.4 Encounter Types 26
4.6.5 Integration Flow 26
5. Customization and Content Creation 26
5.1 Defining New Cards 27
5.1.1 Creating the Asset 27
5.1.2 The Dynamic Inspector 27
5.1.3 A Note on Recipes 28
5.1.4 Special Card Types 28
5.2 Creating New Recipes 29
5.2.1 Basic Recipe Configuration 29
5.2.2 Special Recipe Types 30
5.3 Setting Up Quests 31

5.3.1 Defining a Quest 31
5.3.2 Registering the Quest 32
5.4 Creating Encounters 32
5.4.1 Defining an Encounter 32
5.4.2 Registering Encounters 33
5.4.3 How Spawning Works 33
5.5 Customizing Card & Pack Appearance 33
5.5.1 3D Models & Geometry 33
5.5.2 Texture Guidelines & UV Mapping 34
5.5.3 The Custom Shader System 35
5.5.4 Category-Based Materials 36
5.6 Customizing Board & Environment 36
5.6.1 The Dynamic Board Model 36
5.6.2 The Background & Backdrop 37
5.6.3 Lighting & Atmosphere 37
5.7 Creating New Gameplay Scenes 37
5.7.1 Organization 38
5.7.2 Creating a New Level 38
5.7.3 Configuring Local Managers 38
5.7.4 Finalizing the Scene 39
5.8 Scene Travel & Linking 39
5.8.1 The Travel Recipe 39
5.8.2 How Travel Works 39
5.8.3 Spawning on Arrival 40
5.8.4 Best Practices for Linking 40
6. Third-Party Assets 40
6.1 Coding & Animation Tools 40
6.2 Visual Assets & Icons 41
6.3 Audio & Music 41

1. Introduction

Thank you for adopting **StackCraft**! We appreciate your support and are excited to provide you with a powerful, modular foundation for creating your own card-based survival or management game.

This template is built around a flexible Day Cycle system and robust Card Management, allowing for complex interactions like Crafting, Combat, and Questing.

This documentation will guide you through setup, usage, and customization so you can get the most out of this package and create your own unique card-stacking experience.

1.1 Template Overview

The **StackCraft** template provides a complete foundation for creating a card-based survival, resource management, or simulation game centered around a flexible card-stacking mechanic and a structured time loop.

Inspired by management and deck-building genres, it combines core survival mechanics with robust, modular managers for streamlined development.

Key features include:

- **Modular Management System**: Clear separation of concerns handled by dedicated managers for Cards, Combat, Crafting, Quests, and Trade.
- **Structured Day Cycle**: A flexible system that governs the flow of time and manages phased events, moving from resource management to encounters.
- **Card-Stacking Mechanics**: A core mechanic where players combine resources (cards) to trigger actions, craft items, or prepare for the day.
- **Integrated Game Persistence**: The template provides a structure for saving and loading all card states, stacks, and progression data.
- **Combat System**: A turn-based system complete with a *Rock-Paper-Scissors* (RPS) multiplier for tactical depth.
- **Quest & Progression Tracking**: Use Quest Manager to set up and track various quest types (Craft, Explore, Time) and reward player progression.

This template is designed to be easily extensible for developers who want to add new content, mechanics, or unique card interactions to their project.

1.2 Target Audience

This template is designed for developers who want a fast, organized, and modular starting point for creating their own card-based survival, management, or resource allocation game.

Whether you are:

- **An indie developer** experimenting with complex game loops that involve card-stacking, crafting, and time management.
- **A student** learning advanced Unity architecture through the use of dedicated manager scripts (Singletons) for systems like Combat, Trade, and Quests.
- **A publisher** seeking a polished and extensible foundation to quickly build and release a new management or deck-building title.

StackCraft provides the necessary structure and performance to build and extend content quickly.

1.3 Support and Contact Information

For technical support, feature requests, or general inquiries regarding the **StackCraft** template, please use the following contact details:

- **Publisher:** Crying Snow
- **Website:** https://crying-snow.github.io/
- **Email Support:** cryingsnowstudios@gmail.com
- **Discord Server:** https://discord.gg/avSwFtNEGj

2. Getting Started

This section will guide you through the process of setting up **StackCraft** and getting your project running. We'll cover everything from importing the template to understanding the main scene and core managers.

By the end of this section, you'll be ready to start customizing your card-stacking game.

2.1 Installation & Setup

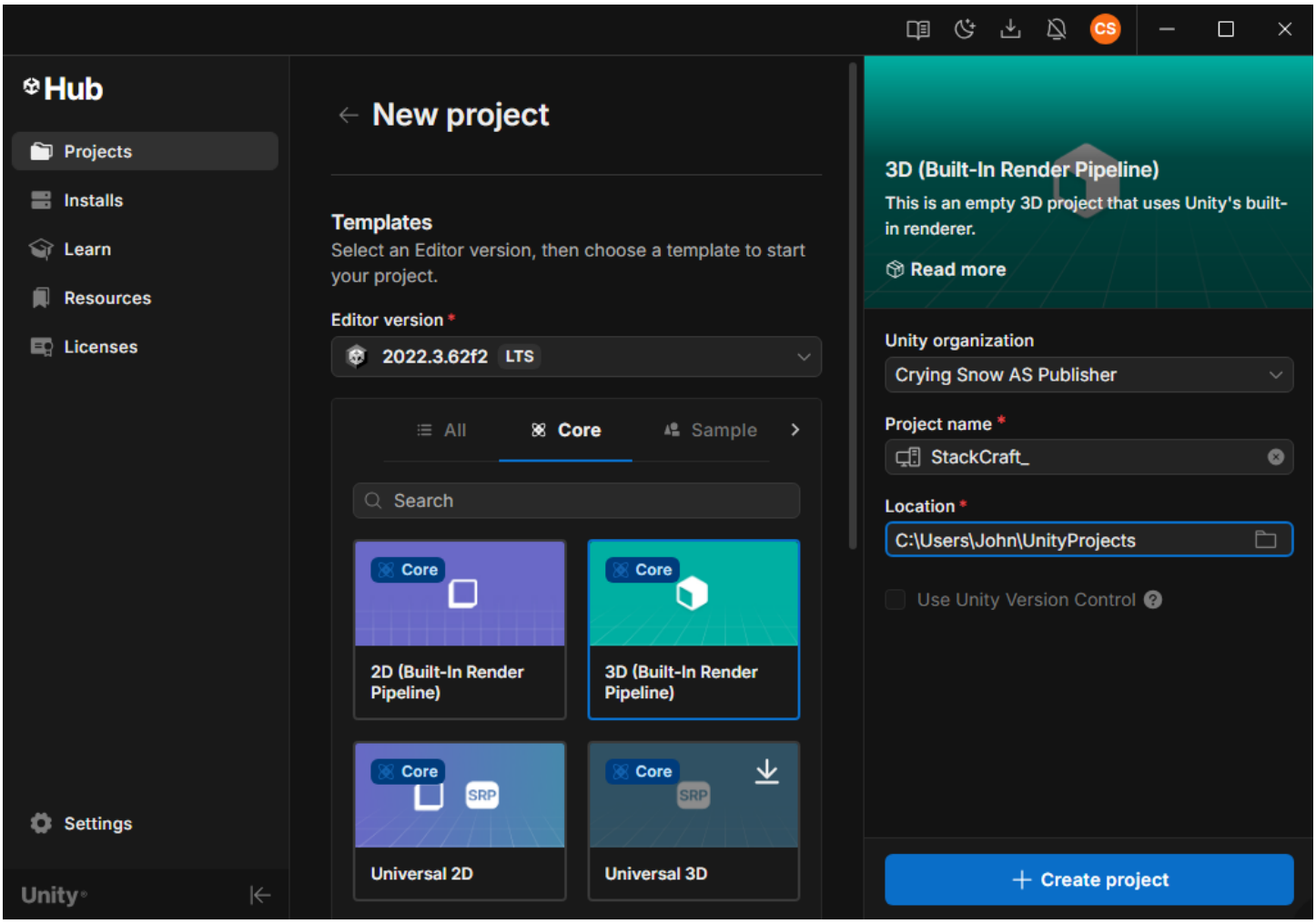
Follow these steps to successfully import and configure the **StackCraft** template in your Unity project.

2.1.1 Prerequisites

- **Unity Version:** The project is developed using **Unity 2022.3.62f2**. Using a different version may lead to minor compatibility issues.
- **Editor Mode: StackCraft** is designed for a single player/single viewport experience, typically in Standalone or PC/Mac build targets.

2.1.2 Importing the Template

1. **New Project:** Create a new Unity project. The **3D Built-in** or **3D URP** template is recommended as a starting point.



2. **Cross-Pipeline Compatibility: StackCraft** is designed to be fully compatible with both the **Built-in Render Pipeline (BRP)** and the **Universal Render Pipeline (URP)**.
    - All materials and shaders utilize **Shader Graph** and are configured to work seamlessly across both pipelines.
    - You can switch pipelines at any time during development without breaking visuals.
3. **Switching Pipelines (Optional):**
    - To switch to URP, navigate to **Tools > Crying Snow > Switch to URP** in the Unity menu.
    - To switch back to the Built-in Pipeline, navigate to **Tools > Crying Snow > Switch to Built-in**.
4. **Import:** Open the Unity Package Manager window (**Window > Package Manager**). In the top-left dropdown, select **"My Assets"** and search for **StackCraft**. Select the package and click **Download**, then **Import**.
5. **Accept:** When prompted by Unity, accept the request to install or update any required packages (dependencies).

2.1.3 Initial Configuration

1. **Scene Check:** Navigate to the `Assets/StackCraft/Scenes` folder, and load the `Main` scene to understand how a gameplay scene is structured.

   Note: Due to the included `PlayModeStartScene.cs` editor script, the `Title` scene will automatically be loaded whenever you enter Play Mode in the Unity Editor, ensuring the project always starts correctly.

2. **Run:** Press Play in the Unity Editor (regardless of the currently active scene). The `Title` scene will load, allowing you to create a new game or load a previously saved one. Creating a new game will automatically load the `Main` scene and initialize all managers.

2.1.4 Important Folders

The `Assets/StackCraft` folder contains all core assets, systems, settings, and content for the **StackCraft** project. Below is an overview of the main folders you will interact with.

Folder	Description
Scenes	All game scenes, including the title screen and gameplay scenes.
Scripts	All runtime and editor scripts that power gameplay systems, UI, and tools.
Resources	Data-driven content loaded at runtime (cards, recipes, packs, quests, encounters).
Prefabs	Reusable prefabs used throughout the project (cards, UI, core systems, VFX).
Materials	Materials used for cards, boards, UI, effects, and backgrounds.
Shaders	Custom shaders and Shader Graph assets (Built-in & URP compatible).
Textures	Textures used for card art, backgrounds, packs, and visuals.
Sprites	2D sprites mainly used for UI and combat feedback.
Models	3D models used by cards, boards, packs, and UI elements.
Sounds	Music, sound effects, and the main audio mixer.
Settings	Project and rendering-related settings, including URP assets.

2.2 Scene Structure

The core gameplay of **StackCraft** is organized within the gameplay scenes (e.g., Main, Island), which is structured to clearly separate persistent system managers, core game elements, and UI components.

2.2.1 Director Layer (Persistent Singletons)

These GameObjects are marked `DontDestroyOnLoad` and persist across scene transitions (e.g., traveling to Island, back to title). They are initialized in the `Title` scene and act as global services.

GameObject	Key Script	Role
GameDirector	`GameDirector.cs`	The persistent singleton that handles game flow, saving/loading, and scene management (e.g., traveling).
AudioManager	`AudioManager.cs`	Manages all sound effects (SFX) and background music (BGM) using an AudioMixer, and handles volume saving.
GraphicsManager	`GraphicsManager.cs`	Manages graphics settings, including resolution, fullscreen mode, VSync, and dynamic shadow quality adjustments (especially for URP).

2.2.2 Gameplay Scene Hierarchy

The remaining GameObjects are contained within the gameplay scenes (e.g., Main, Island) and define the visual and interactive components of the current game area.

GameObject	Key Script	Role
CameraController	`CameraController.cs`	Controls the user's ability to pan and zoom. Also applies visual effects like the **grayscale/vignette effect** during certain game states (e.g., day end or paused).
Directional Light	N/A	The main light source (from default scene creation).
Background	N/A	The visual backdrop for the scene. A simple plane object with tiling texture applied.
Board	`Board.cs`	The physical game plane. It defines the world bounds where cards can be placed and manages the `Top Margin` to reserve space for the trade zones.
GameManagers	`All *Manager.cs`	This is a container object holding all in-scene manager Singletons that handle core game logic.
UIRoot	N/A	The parent object for all User Interface elements, containing the UI Canvas, World Canvas (for elements like progress bars above cards), and the Event System.

2.2.3 Manager Organization (GameManagers Children)

All non-persistent game logic is organized as children of the GameManagers GameObject, adhering to the Singleton pattern for easy access (`*Manager.Instance`).

Manager	Functionality
CardManager	Core logic for card creation, destruction, stacking, and player stats (Nutrition, Currency, Card Limit).
CraftingManager	Manages recipes, tracks active crafting/exploration tasks, and handles recipe discovery.
TradeManager	Manages the **Card Buyer** (selling cards) and **Pack Vendor** (buying packs) trade zones.
CombatManager	Handles the initiation, execution, and resolution of combat encounters, including the *Rock-Paper-Scissors* (RPS) rule.
QuestManager	Manages QuestGroups, tracks player progress, and handles quest completion rewards.
TimeManager	A global manager responsible for in-game time flow, day progression, and pause control.
DayCycleManager	Orchestrates the day-to-day phases: Feeding, Selling Excess, Encounter Execution, New Day setup, and Game Over.
EncounterManager	Evaluates and executes time-based events and card spawns based on `EncounterDefinitions`.
InputManager	Global input handling, managing input locks during key events (like the end of the day cycle).

2.3 Game Persistence

StackCraft uses a robust system for saving game state across play sessions and for managing temporary data during scene transitions (e.g., when the player "travels"). This system is primarily handled by the persistent `GameDirector` and the serializable data structure, `GameData.cs`.

2.3.1 The GameData Structure

All critical game state information is encapsulated within the `GameData` class, which is saved to the disk (serialized).

Class Name	Data Stored	Relevant Manager
GameData	Root data container. Stores SlotNumber, CurrentScene, GameplayPrefs, discovered cards/recipes, and a dictionary of **SavedScenes**.	`GameDirector.cs`
SceneData	Stores all scene-specific data, including **SavedStacks**, active CombatData, active/completed quests, and VendorData.	`GameDirector.cs`
TimeData	Stores the current state of the in-game clock: CurrentTime (time of day) and CurrentDay.	`TimeManager.cs`
StackData	The most critical data structure. It serializes a complete CardStack, including its world Position, the list of CardData it contains, and any active CraftingTaskData.	`CardManager.cs`, `CraftingManager.cs`
CardData	Stores the serializable information for a single card: its DefinitionID, InstanceID, Health, and current Faction.	`CardManager.cs`
QuestData	Stores the ID of an active quest and its current CurrentAmount of progress.	`QuestManager.cs`

2.3.2 Saving and Loading Flow

The saving process is handled automatically by the `GameDirector` and ensures all managers contribute their current state.

1. **Request Save:** The `GameDirector.Instance.SaveGame()` method is called (e.g., at the end of a day or when quitting app).
2. **Manager Contribution:** The `GameDirector` invokes the `OnBeforeSave` event. All managers that hold serializable data (like `CardManager`, `QuestManager`, `Encounter Manager`) listen to this event and populate the current `SceneData` with their state.
3. **Serialization:** The final `GameData` object is serialized (converted to JSON file) and written to a file slot on the disk.

4. **Loading:** When a game is loaded:
    - The `GameDirector` deserializes the `GameData` from the chosen slot.
    - The `GameDirector` loads the `CurrentScene`.
    - Once the scene is loaded, the `GameDirector` invokes the `OnSceneDataReady` event, providing the `SceneData` to all managers to restore their state (e.g., `CardManager` reconstructs all card stacks, and `QuestManager` restores active quests).

2.3.3 Scene Travel

The `GameDirector` also manages the travel feature (e.g., moving between `Main.unity` and `Island.unity`):

- When traveling, the `GameDirector` first saves the current scene's state.
- The cards designated as travelers are temporarily serialized as `CardData` and passed to the next scene.
- Upon loading the new scene, the `GameDirector` restores the new scene's saved data and then spawns the incoming travelers into the new location.

3. Core Game Loop: The Day Cycle

The core experience of **StackCraft** is built around a structured, multi-phase **Day Cycle**. This system, controlled primarily by the `DayCycleManager.cs`, dictates when the player must manage resources, when new threats or opportunities appear, and when the game state is saved.

The entire process begins when the in-game clock (managed by `TimeManager`) signals the end of the current day.

3.1 Overview of Time

Phase Duration	Script Control	Description
Real-time	`TimeManager`	During the day, time flows normally, allowing the player to freely stack cards, craft, and trade.
Phase-based	`DayCycleManager`	When the day ends, the `DayCycleManager` takes control, locking player input (`DayCycleInputLock`) and processing the end-of-day sequence phase by phase.

3.2 Phase 1: Notification

This initial phase serves as a brief interlude between real-time play and the management phases.

1. **Input Lock:** Player input is locked using the `DayCycleInputLock`.
2. **Day End Display:** A modal panel appears (handled by `InfoPanel`) informing the player that the day has ended.
3. **Next Phase Trigger:** Upon dismissing the notification, the system proceeds to the core Resource Management phase.

3.3 Phase 2: Resource Management (Feeding & Selling)

This phase ensures the player has enough sustenance and is within their card limits before the new day can begin.

3.3.1 Feeding Check (Survival)

This phase represents the core survival loop of **StackCraft**. Instead of a single global comparison, feeding is resolved per character, one at a time, using available consumable cards on the board.

Overview

- All character cards are processed sequentially.
- Each character attempts to satisfy their hunger by consuming nearby consumable cards.
- Feeding is spatially aware and animated, making survival outcomes visible and deterministic.

Feeding Process

For each character card:

1. **Camera Focus (Presentation)**

   The camera moves to the character being processed to clearly show the feeding outcome.

2. **Hunger Initialization**

   Each character starts with a fixed hunger value (`HungerPerCharacter` defined in `CardSettings`).

3. **Consumable Selection**

   While the character is still hungry, the system searches for the **nearest consumable card** with remaining nutrition.

4. **Consumption & Healing**

   The selected consumable transfers nutrition to the character:

   - Hunger is reduced by the consumed nutrition amount.
   - The character is healed up to **50% of max health** if fully fed.
   - Consumables with remaining nutrition stay on the board and can be reused.

5. **Death Condition**

   If no consumable cards remain while the character is still hungry:

   - The character immediately dies (card is destroyed).
   - Feeding for that character stops.

3.3.2 Selling Check (Card Limit)

This phase forces the player to manage their inventory down to the allowed limit.

1. **Card Limit Check:** The system retrieves the current card count (`CardsOwned`) versus the maximum allowed (`CardLimit`), which is derived from `LimitBooster` cards (e.g., yard, warehouse).
2. **Excess Card Penalty:** If the player's `CardsOwned` exceeds their `CardLimit` (`ExcessCards > 0`), the game enters a wait state:
   - The `DayCycleManager` displays a modal panel, forcing the player to **sell** excess cards to clear space.
   - The manager awaits a satisfied condition.

3.4 Phase 3: Encounters & Events

After the survival and resource phases complete, the game enters the Encounter Phase, where at most one event-driven encounter may occur for the current day.

3.4.1 Encounter Selection

The `EncounterManager` selects the single most appropriate encounter for that day. If no valid encounter exists, the phase is skipped entirely and the game proceeds to the next day.

3.4.2 Encounter Execution

If an encounter is selected:

1. **Input Lock**
   Player input is temporarily locked to prevent interaction during encounter execution.

2. **One-Time Encounter Tracking**
   If the encounter is marked as *One Time Only*, it is recorded as completed and will not trigger again in future days.

3. **Event Notification**

   If the encounter defines a notification message:

   - A modal information panel is displayed to the player.
   - The game waits briefly to ensure the message is clearly visible.

4. **Card Spawning**

   The encounter spawns a predefined number of cards:

   - Each card is spawned at a random valid board position.
   - Cards may represent threats or neutral events depending on the definition.

   For each spawned card:

   - The camera moves to the spawn location to highlight the event.
   - Visual effect (e.g., puff particle) is played.

5. **Cleanup**

   - Encounter-related UI notifications are cleared.
   - Player input is restored once execution finishes.

3.5 Phase 4: New Day

This is the final sequence that completes the loop and prepares the game for the next day of player activity.

1. **Start of Day Notification:** A final modal panel is displayed to the player, confirming the start of the next day (`Start of Day X`).
2. **Finalization & Save:** When the player confirms the notification:

   - The `IsEndingCycle` flag is reset to false.
   - `TimeManager.Instance.StartNewDay()` is called to increment the day counter and resume time flow.
   - `GameDirector.Instance.SaveGame()` is called to save the entire new game state to the current slot.

3.6 Game Over Condition

The `DayCycleManager` checks for a Game Over state during the cycle: if the player has **no people left** on the board, a "Game Over" modal is displayed, allowing the player to return to the title scene via `GameDirector.Instance.GameOver()`. This also clears the active save file.

4. Game Systems Deep Dive

This section details the core systems built into **StackCraft**, explaining how cards are managed, how the stacking mechanic works, and the processes behind crafting, combat, and quests.

4.1 Card & Stack Management

The **Card Management System** is the foundation of **StackCraft**. It controls the lifecycle of every card, from spawning and stacking to tracking stats and destruction. This logic is centralized in `CardManager.cs` and relies on two main objects: `CardInstance.cs` (the visual card) and `CardStack.cs` (the structural container).

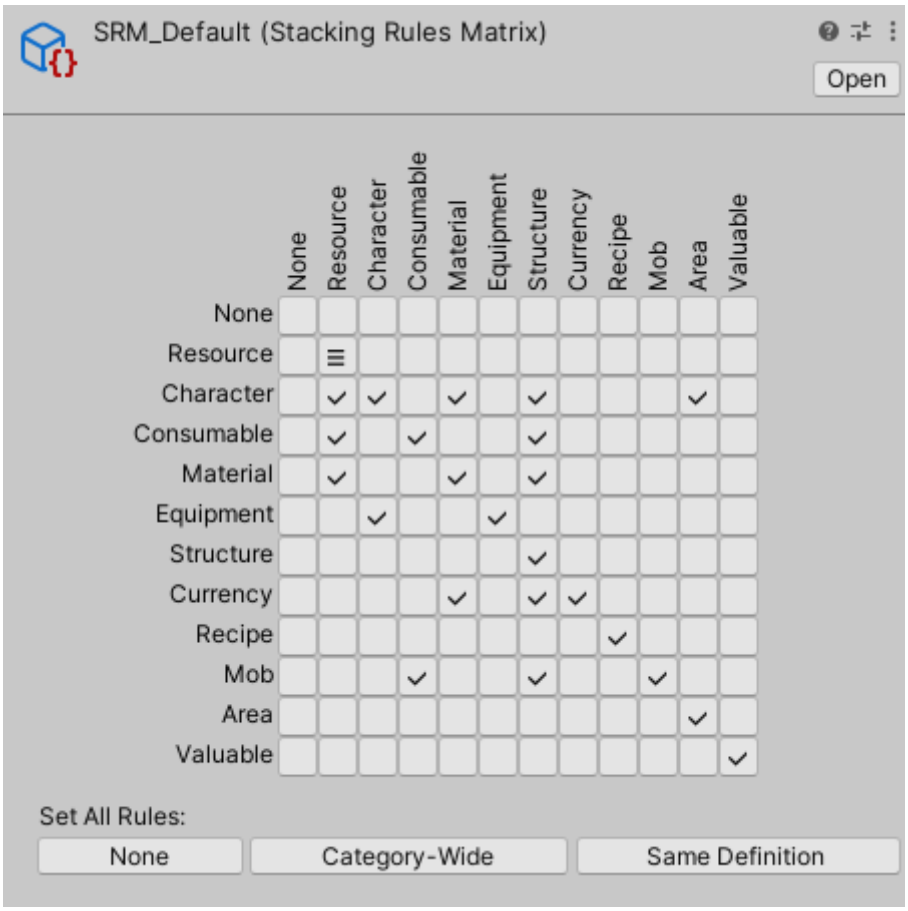
4.1.1 Card Stacking Main Components

Script	Role	Description
`CardInstance.cs`	Visual Object	The MonoBehaviour attached to the physical card prefab. It handles all visual updates, displaying Health, Nutrition, and Price. It holds a reference to its immutable `CardDefinition` (Scriptable Object).
`CardController.cs`	Input Handler	Manages player interactions (picking up, dragging, dropping). It is responsible for checking valid drop targets (stacks, trade zones, or combat areas).
`CardStack.cs`	Data Structure	A container class that manages a vertical stack of one or more `CardInstance` objects. It defines the stack's world position, ensures the cards are correctly offset, and applies custom physics translations.

4.1.2 The Stacking Mechanic

Stacking is the primary method for players to combine resources, unlock new functions, and manage space.

- **Stacking Rules:** The validity of stacking is determined by the `StackingRulesMatrix.asset` (a `ScriptableObject`). This matrix defines the stacking rule between any two `CardCategory`.
  - **Rule Types:**
    - **None**: No stacking is allowed between these two categories.
    - **CategoryWide (✓)**: Any card in the **top** category can be stacked on any card in the **bottom** category (e.g., any Character on any Resource).
    - **SameDefinition (≡)**: Only cards of the exact same `CardDefinition` can be stacked (e.g., Tree on Tree).



- **Process:** When a player drops a card (`CardController.OnPointerUp`):
  1. The system checks for nearby `CardStack` objects.
  2. It uses the `StackingRulesMatrix` to determine if placing the dragged card (`TopCard`) onto the existing stack (`BottomCard`) is valid.
  3. If valid, `CardManager` moves the card from its old stack (or unstacked state) to the new `CardStack`, and all card positions are recalculated and animated.

4.2 Crafting & Industry

Crafting is the core loop of **StackCraft**. By stacking cards together, you combine resources, materials, and villagers to create new items, structures, or discoveries. This section outlines how recipes are detected, processed, and executed.

4.2.1 The Basics: How to Craft

To initiate crafting, simply **stack two or more compatible cards** on top of each other.

- **Automatic Detection:** The game automatically scans the stack to see if the combination matches any known `RecipeDefinition`.
- **Weighted Probability:** If a stack matches *multiple* valid recipes, the game chooses one based on a *weighted random system*. Recipes with a higher `RandomWeight` are more likely to occur.

- **The Progress Bar:** Once a valid recipe is found, a `ProgressUI` bar appears above the stack. You must wait for the timer (`CraftingDuration`) to complete.
- **Pause/Resume:** If you modify the stack mid-craft, the process pauses. If the new stack is still valid (e.g., you added more Wood to a Sawmill), it resumes. If the stack is invalid, the craft is canceled.

4.2.2 Recipe Categories

A. Standard Crafting (Strict Matching)

Most basic recipes require an **exact match** of ingredients.

- **Rule:** The stack must contain *exactly* the cards listed in the recipe.
- **Example:** If a recipe requires 1 Wood + 1 Stone, stacking 2 Wood + 1 Stone will **not** trigger the recipe. You must split the stack to match the requirement exactly.

B. Workstations & Continuous Crafting

Certain structures (like Logging Camp, Quarries) utilize **Continuous Crafting**.

- **Allow Excess Ingredients:** Unlike standard crafting, these recipes function even if you stack *more* ingredients than necessary.
    - *Example:* A Sawmill requires 1 Wood. You can stack 10 Wood on it, and it will process them one by one.
- **Live Feeding:** You can add cards to a workstation *while* it is working. The `CraftingManager` allows new cards to join the stack if they are valid ingredients for the current task.
- **Auto-Repeat:** Once a workstation finishes a task, it immediately checks if it can craft again using the remaining cards in the stack.

4.2.3 Special Recipe Types

Beyond basic item creation, **StackCraft** includes specialized recipe logic defined in the code:

Recipe Type	Behavior Description
Growth	Used for farming. Requires a Grower (e.g., Planter Box) and a Seed. When complete, the seed splits off into a new stack and transforms into the resulting crop, leaving the Grower behind.
Exploration	Requires an Area card (e.g., Forest) and a unit (e.g., Villager). Does not consume the cards but spawns random loot defined by the Area's loot table.
Research	Used to unlock new technology. Requires certain amount of characters to spawn a Recipe Card for a random, currently undiscovered recipe.
Travel	Moves character card to a completely different game scene (e.g., entering an Island or a Dungeon).

4.2.4 Ingredient Consumption

Not all ingredients are treated equally. When a recipe finishes, ingredients are processed according to three specific modes:

1. **Consume:** The card's "Uses" or "Durability" is reduced by 1. If it reaches 0, the card is destroyed.
2. **Destroy:** The card is removed immediately, regardless of durability.
3. **Keep:** The card is untouched.

4.2.5 Recipe Discovery

You do not need to "know" a recipe to use it, experimenting with stacks is part of the gameplay. Any valid combination will resolve correctly when the required cards are stacked together, even if the player has never encountered that recipe before.

- **Discovery Trigger:** When you successfully complete a craft for the first time, or use a *Research Recipe*, that recipe is permanently added to your `DiscoveredRecipes` list.
- **Recipe Cards:** Sometimes you will find "Recipe Cards" as loot. These are reference cards that show you the ingredients required for a specific item.

4.3 Trading & Economy

Trading is the primary method for converting your excess resources into currency and purchasing new card packs to expand your possibilities. The trading row is located at the top of the board and consists of distinct Trade Zones.

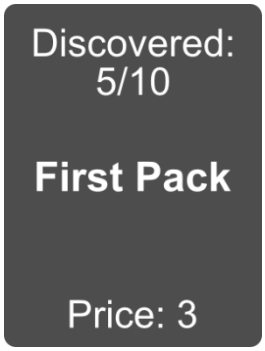
4.3.1 Selling Cards (The Card Buyer)

The **Card Buyer** is your main source of income. It is always available and is marked by the currency icon used in the current scene (for example, a **Coin**).

- **How to Sell:** Drag a stack of cards onto the `CardBuyer` zone.
- **Payout:** The buyer calculates the total `SellPrice` of every card in the stack. The cards are destroyed, and an equivalent number of **currencies** are spawned below the zone.
- **Restrictions:**
    - **Sellable Only:** Cards marked as unsellable in their definition cannot be sold.
    - **Empty Chests Only:** You cannot sell a Chest if it still contains coins. You must empty it first.

4.3.2 Buying Packs (Pack Vendors)

Pack Vendors allow you to purchase new cards using currency. Each vendor sells a specific type of Card Pack (e.g., "Starter Pack", "Nature Pack").



- **How to Buy:** Drag a stack of **Coins** or a **Chest containing Coin** onto the vendor.
- **Smart Payment:**
  - **Partial Payment:** You do not need to pay the full price at once. If a pack costs 10 Coin and you drop 3 Coin, the vendor remembers you paid 3. The price text updates to show the remaining cost (e.g., "Price: 7").
  - **Chests:** If you drop a Chest on a vendor, it will automatically withdraw coins from the chest one by one until the pack is paid for or the chest is empty.
- **Delivery:** Once the full price is paid, a `PackInstance` spawns on the board below the vendor.

4.3.3 Unlocking Vendors

Not all packs are available at the start of the game.

- **Quest Progression:** New vendors are locked behind `Quest` milestones. A vendor will only activate and become visible once you have completed the required number of quests (`MinQuests`).
- **Activation Sequence:** When a new vendor unlocks, the game pauses, the camera focuses on the new trade zone, and an "Unlocked" notification appears.

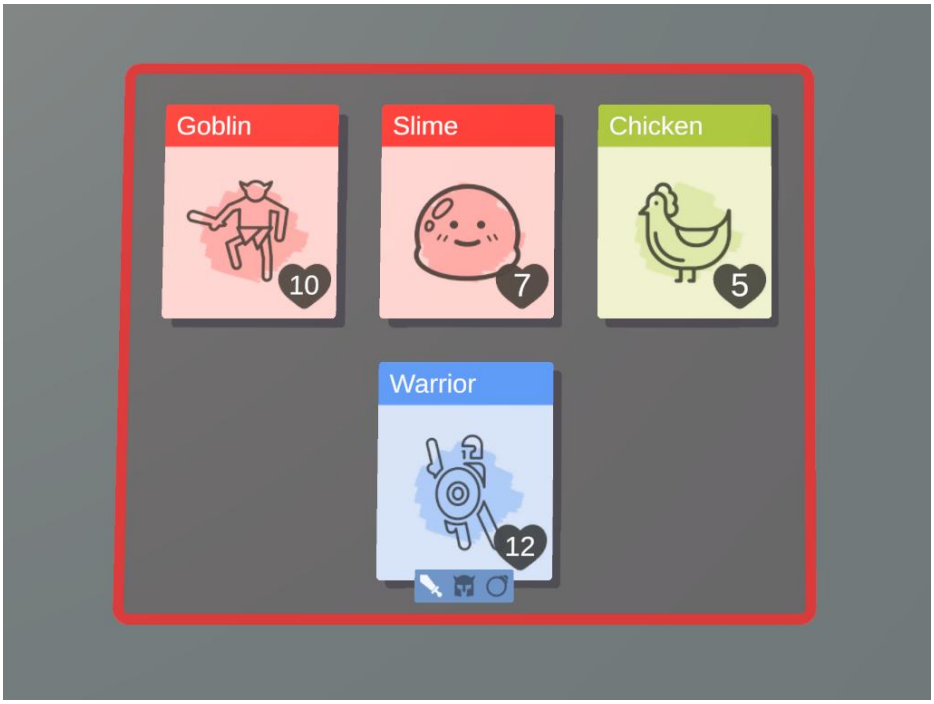
4.3.4 Collection Tracker

Each Pack Vendor features a built-in progress tracker to help completionists.

- **Tracking:** The vendor displays a fraction (e.g., "Discovered: 5/12") indicating how many unique **Cards** and **Recipes** from that specific pack you have discovered.
- **Completion:** Once you have discovered every possible item and recipe contained in that pack, the text turns gold and displays **"COMPLETED"**.

4.4 Combat & Conflict

Combat occurs whenever opposing factions meet on the board. Unlike crafting, which is peaceful, combat is a high-stakes struggle that can result in the permanent loss of your villagers.



4.4.1 Initiating Combat

Combat is triggered by **proximity** or **stacking**:

- **The Aggressive Stack:** If you drag a Player unit (e.g., a Villager) onto a Mob unit (e.g., a Slime), combat begins immediately.
- **The Combat Arena:** When combat starts, a **Combat Rectangle** (`CombatRect`) appears on the board. All participating units are moved into this area, organized into two rows: **Attackers** and **Defenders**.
- **Reinforcements:** You can drag additional combat-capable cards into an ongoing Combat Rectangle to join the fight.

4.4.2 Combat Attributes (Stats)

Every unit has a set of `CombatStats` that determines their effectiveness. You can view these by hovering over a card:

- **Health (HP):** The unit's life force. If it hits zero, the card is destroyed.
- **Attack:** The base damage dealt to an enemy.
- **Defense:** Reduces incoming damage from enemy attacks.
- **Attack Speed:** Determines how quickly the unit's "Action Bar" fills up.
- **Accuracy vs. Dodge:** Determines the chance of an attack being a "Miss."
- **Critical Chance & Multiplier:** The probability and power of a high-damage "Critical Hit."

4.4.3 The Combat Loop

Combat is **semi-automated** and follows a specific rhythm:

1. **Action Progress:** Each unit has an internal timer. Once it reaches 100%, the unit performs an attack.
2. **Targeting:** Units target random enemy. If that enemy is defeated, they shift their focus to the next available target.
3. **Attack Types:**
    - **Melee:** The unit physically bumps into the target.
    - **Ranged & Magic:** The unit launches a projectile (e.g., arrow, spell).

4.4.4 Types & Advantages (Rock-Paper-Scissors)

The **StackCraft** combat engine uses an **Effectiveness System** to reward tactical positioning. Using a unit with a type advantage grants a **Damage Multiplier** (e.g., 1.5x damage).

- **Advantage:** Deals more damage and shows an "Advantage" icon in the Hit UI.
- **Disadvantage:** Deals reduced damage (e.g., 0.75x) and shows a "Disadvantage" icon.
- **Feedback:** The `HitUI` will display damage numbers: Orange for normal hits, Red for Criticals, and **"Miss"** for failed attacks.

4.4.5 Automated AI & Aggression

Units behave differently based on their `CardAI` settings:

- **Aggressive Units:** These units will actively look for targets. If an aggressive mob is free on the board, it may move toward your villagers to force a fight.
- **Passive Units:** Some units will never initiate combat but will defend themselves if attacked.
- **Production during Peace:** Many units (like Chickens or Cows) have a "Produce Loop" that creates items (like Eggs) over time. However, production stops the moment the unit enters combat.

4.4.6 Ending Combat

Combat ends under two conditions:

1. **Victory/Defeat:** One side is completely wiped out. The remaining units "Leave Combat" and return to their normal AI behavior (moving or producing).

2. **Retreat:** Combat can only be ended manually when the player initiates the combat (`PlayerIsAttacker` flag). In this case, the player may drag one or more involved combatant cards out of the combat rectangle to disengage and end the combat.

4.5 Quest System

The Quest System in **StackCraft** acts as the primary driver for game progression. It tracks player actions to unlock new narrative steps and expand the marketplace (Packs).

4.5.1 Quest Structure

A `Quest` is defined by a `ScriptableObject` containing descriptive data and logic requirements.

- **Title & Description:** The display text shown to the player.
- **Target:** The specific object (Card or Recipe) required to satisfy the quest.
- **Amount:** The quantity required (e.g., "Craft 5 sticks").
- **Prerequisites:** A list of quests that must be completed before this one becomes active.

4.5.2 Progression & Unlocks

Quests function on a **dependency chain** system:

1. **Inactive:** The quest is hidden and not tracking progress because prerequisites are not met.
2. **Active:** Once all `PrerequisiteQuests` are complete, the quest automatically activates and begins listening for game events.
3. **Completed:** When the `TargetAmount` is reached, the quest is marked complete, and it immediately triggers any quests listed in `QuestsToUnlock`.

4.5.3 Quest Types & Objectives

The system supports several distinct categories of objectives. It is important to understand the difference between **State Checks** (Current Status) and **Action Checks** (Cumulative Events).

A. Inventory & State (Current Status)

These quests check the *current* state of the player's board. Progress can fluctuate up and down.

- **Have:** Requires the player to currently hold a specific amount of a Card.
- **Coins:** Requires the player to reach a specific currency balance.
- **Food:** Requires reaching a total nutrition value (sum of all consumable cards).
- **Capacity:** Requires increasing the `CardLimit` (by building `LimitBooster` cards).

B. Actions (Cumulative)

These quests track how many times an action has been performed. Progress implies "Lifetime Total" and cannot decrease.

- **Obtain:** Triggers when a new card is created/spawned.
- **Craft:** Triggers specifically when a card is the result of a crafting recipe.
- **Buy:** Triggers when purchasing Packs from the market.
- **Sell:** Triggers when selling cards. Supports "Sell specific card" or "Sell X amount of any card".
- **Equip:** Triggers when equipping an item to a character/villager.
- **Explore:** Triggers when an exploration action finishes on a specific Area card.
- **Defeat:** Triggers when an enemy card is killed/destroyed.

C. Knowledge & Time

- **Discover:** Triggers when a new `RecipeID` is added to the player's knowledge base.
- **Day:** Triggers when the game reaches a specific Day number.
- **Time:** Triggers when the player changes the game speed (Pace).

4.6 Encounter System

The Encounter System controls the procedural generation of events, enemies, and resources. It operates as a distinct phase within the daily game loop, evaluating game state conditions to inject new cards onto the board dynamically.

4.6.1 Overview

The system is managed by the `EncounterManager` singleton. At the end of every in-game day (after feeding and selling phases), the manager evaluates a list of `EncounterDefinition` assets to determine if an event should trigger.

Key Responsibilities:

- **Selection:** Filtering and prioritizing potential events based on the current day, game mode, and randomness.
- **Execution:** Handling the visual sequence (camera movement, notifications, particle effects) of spawning cards.
- **Persistence:** Tracking "One-Time-Only" events to ensure they do not repeat across a save file.

4.6.2 Core Architecture

A. Encounter Manager

The central processor for events. It listens to `GameDirector` events (`OnSceneDataReady`, `OnBeforeSave`) to persist the state of completed encounters.

- **Spawn Logic:** Cards are spawned at random positions within the board bounds, respecting a configurable `spawnEdgePadding` to prevent cards from clipping outside board.
- **Camera Integration:** The manager temporarily takes control of the `CameraController` to focus on the spawn location during an event.

B. Encounter Definition

A `ScriptableObject` asset that defines the rules for a specific event.

Property	Description
Id	A unique GUID used to track completion status. Automatically generated on validation.
Type	Defines the timing logic (Specific Day, Recurring, Range, or Minimum Day).
Priority	Integer value used to resolve conflicts when multiple encounters are valid simultaneously.
Chance	A normalized float (0.0–1.0) representing the probability of the event triggering if conditions are met.
OneTimeOnly	If true, the event ID is added to `completedEncounters` upon execution and will never trigger again.

4.6.3 Selection Logic

When `GetBestEncounter(day)` is called, the system performs a two-step filtration process to select a single event.

Step 1: Validation

An encounter is valid only if **all** the following are true:

1. **Board Limit:** Total cards on board less than `maxCardsOnBoardLimit`.
2. **Completion:** If `OneTimeOnly` is set, the ID must not be in the `completedEncounters` list.
3. **Game Mode:** If `IsFriendlyMode` is active, encounters with `Aggressive` cards are ignored.
4. **Timing:** The current day matches the `EncounterType` criteria (see below).
5. **RNG:** `Random.value` satisfies the `chance` threshold.

Step 2: Prioritization

If multiple candidates pass validation, they are sorted hierarchically:

1. **User Priority:** Descending order (Higher Priority wins).
2. **Intrinsic Logic:** If priorities are equal, the `EncounterType` determines the winner in this specific order:

1. *Specific Day* (Highest)
2. *Recurring*
3. *Range*
4. *Minimum Day* (Lowest).

4.6.4 Encounter Types

The system supports four distinct timing behaviors via the `EncounterType` enum:

- **SpecificDay:** Triggers exactly on the defined `DayValue` (e.g., a tutorial event on Day 2).
- **Recurring:** Triggers whenever `CurrentDay % DayValue == 0` (e.g., a mob every 7 days).
- **Range:** Triggers between `DayValue` and `MaxDayValue` inclusive.
- **MinimumDay:** Triggers on any day greater than or equal to `DayValue`.

4.6.5 Integration Flow

The `DayCycleManager` orchestrates the encounter phase as part of the end-of-day sequence:

1. **Trigger:** After the "Feeding" and "Selling" phases are complete, `DayCycleManager` calls `EncounterManager.GetBestEncounter`.
2. **Input Lock:** If an encounter is found, `InputManager` locks player input to prevent interference during the sequence.
3. **Execution:**
    a. The `InfoPanel` displays the `NotificationMessage`.
    b. The camera pans to the spawn location.
    c. Cards are instantiated with a "puff" particle effect.
4. **Resume:** Once the sequence finishes, input is unlocked, and the "Start New Day" UI is presented.

5. Customization and Content Creation

StackCraft is designed to be highly data-driven. The vast majority of gameplay content like items, mobs, structures, and resources, can be created entirely within the Unity Editor without writing a single line of C# code.

The game uses ScriptableObjects to define the properties and behaviors of game entities. This allows designers to rapidly iterate on balance, drop rates, and combat stats.

5.1 Defining New Cards

In **StackCraft**, every entity on the board is defined by a `CardDefinition`. Whether it is a raw material like Wood, a complex structure like a House, or an enemy Mob, they all start as a `CardDefinition` asset.

5.1.1 Creating the Asset

To create a new card, navigate to your Project window.

Note: All Card Definitions must be placed inside the `Resources` folder (or a subdirectory within it) to be loaded correctly by the game systems. We recommend organizing them into subfolders matching their categories (e.g., `Resources/Cards/Mobs`).

- Right-click in the Project window.
- Navigate to **Create > StackCraft**.
- Select **Card** (for standard items/mobs) or one of the **Special Cards** (detailed in Section 5.1.4).

5.1.2 The Dynamic Inspector

The Card Inspector is context-sensitive. Based on the **Category** you select, the Inspector will hide or show relevant fields to keep the interface clean.

Below is a breakdown of the primary configuration sections:

1. Identification

- **ID:** A unique identifier automatically generated when the asset is created.
- **Display Name:** The name shown to the player in the UI.
- **Description:** Flavor text or utility info shown in tooltips.
- **Art Texture:** The visual representation of the card.

Important: Ensure the Texture Import Settings for your art have the Alpha Source set correctly to avoid transparency issues.

2. Classification

- **Category:** This is the most critical field. It determines gameplay behavior (e.g., `Consumable` allows eating, `Equipment` allows equipping).
- **Faction:** (Visible for Character/Mob) Defines allegiance (Player, Mob, Neutral).
- **Combat Type:** (Visible for Character/Mob) Used for *Rock-Paper-Scissors* combat logic (Melee, Ranged, Magic).

3. Loot & Drops *(Visible only for Mobs, Characters, or Areas)* This defines what this card drops when destroyed or explored.

- **Weights:** You can add multiple items with different weights.
- **Normalize Button:** Click **"Normalize Weights (to 100%)"** to automatically adjust the integer weights to percentages.

4. Entity Behavior (Mobs) *(Visible only for Category: Mob)*

- **Aggressive Mob:** If checked, the mob will chase the player. You can define the `AggroRadius` (detection range) and `AttackRadius` (combat range).
- **Passive Mob:** If unchecked, the mob will wander. You can configure it to produce items over time (e.g., a Chicken producing an Egg) by setting the **Produce Card** and **Produce Interval**.

5. Economy & Crafting

- **Trading:** Check `IsSellable` to allow the card to be sold for Gold/Coins.
- **Durability:** If enabled, the card acts like a tool or finite resource (e.g., a Tree with 3 chops). If disabled, it is a single-use item or infinite source depending on context.

6. Stats & Combat *(Visible only for Characters and Mobs)* Here you define the combat capabilities, including `MaxHealth`, `Attack`, `Defense`, and `AttackSpeed`.

- **Crit Settings:** You can also fine-tune `CriticalChance` and `CriticalMultiplier`.

7. Equipment *(Visible only for Category: Equipment)*

- **Slot:** Defines where the item goes (Weapon, Armor, Accessory).
- **Stat Modifiers:** A list of stats this item buffs when equipped (e.g., +5 Attack).
- **Class Change:** (Weapons only) You can link a `ClassChangeResult` to transform the Villager into a new class (e.g., Equipping a Bow transforms a Villager into an Archer).

5.1.3 A Note on Recipes

Warning: Do **NOT** create Recipe category cards manually in the inspector. Recipe cards are generated automatically by the runtime system based on `RecipeDefinitions`. There is one `CardDefinition` asset with Recipe as its category, but it is used by the `CardManager` as a template to create *dynamic instances* at runtime.

5.1.4 Special Card Types

Some cards require specialized logic and have their own creation menu entries. These inherit all standard properties but add specific fields:

Card Type	Special Properties
Grower	Used for farming/growth logic (behavior defined by class type).
Research	Used for unlocking undiscovered recipes (behavior defined by class type).
Enclosure	**Capacity:** Limits how many non-aggressive mobs can stack here.
Limit Booster	**Boost Amount:** Increases the player's total card cap.
Chest	**Capacity:** Defines how much currency (Coins/Corals) it can hold.

5.2 Creating New Recipes

While Cards define the *entities* in **StackCraft**, Recipes define the *interactions*. A recipe tells the game what happens when specific cards are stacked together.

Like Cards, Recipes are ScriptableObjects. The `CraftingManager` loads all recipes located in the `Resources/Recipes` folder at runtime to build the crafting database.

5.2.1 Basic Recipe Configuration

To create a standard recipe:

1. Right-click in the Project window.
2. Navigate to **Create > StackCraft > Recipe**.
3. Place the asset inside `Resources/Recipes`.

The Recipe Inspector exposes several key behaviors:

1. **Ingredients & Consumption**

   You define a list of **Required Ingredients**. For each ingredient, you must specify the `ConsumptionMode`, which dictates what happens to that card when the craft finishes:

   - **Consume:** The default. Uses up durability or stack count. If the card runs out of uses, it is destroyed. Used for resources (Wood, Stone).
   - **Keep:** The card is required but not modified. Used for "Catalysts" like Soil, or a Villager performing a task.
   - **Destroy:** Forces the card to vanish immediately, ignoring any remaining durability.

2. **Output & Timing**

   - **Resulting Card:** The card spawned upon completion.
   - **Crafting Duration:** Time in seconds to complete the bar.

3. **Matching Logic (Strict vs. Workstation)**

   The `AllowExcessIngredients` toggle changes how the game detects this recipe:

   - **Strict (False):** The stack must match the ingredients *exactly*. Use this for combining items (e.g., 1 Wood + 1 Stone = Spear). If a third card is added, the recipe breaks.

- **Workstation (True):** The stack must contain *at least* the ingredients. Use this for buildings (e.g., Sawmill + Wood). This allows the player to stack 10 Wood on a Sawmill; the recipe will process 1 Wood and leave the other 9 waiting.

4. **Automation**
    - **Is Continuous:** If checked, the recipe attempts to run again immediately after finishing, provided ingredients are still present. Essential for "Producer" buildings like a Logging Camp.
5. **Probability**
    - **Random Weight:** If a stack matches multiple valid recipes, the game performs a weighted random roll to decide which one executes. Higher numbers = higher chance.

5.2.2 Special Recipe Types

Complex gameplay mechanics, like farming or research, use specialized recipe subclasses. These are created via their own context menu entries:

1. Exploration Recipe

- **Menu:** `Create > StackCraft > Special Recipes > Exploration Recipe`
- **Usage:** Used for exploring locations (e.g., Plains + Villager).
- **Logic:** It does not spawn a fixed `ResultingCard`. Instead, it looks for an ingredient with the **Area** category and asks that card for its specific Loot Table to generate a random reward.

2. Growth Recipe

- **Menu:** `Create > StackCraft > Special Recipes > Growth Recipe`
- **Usage:** Used for farming (e.g., Soil + Seed).
- **Logic:** It separates the `ResultingCard` (the crop) into a new stack next to the plot, ensuring the soil remains free for replanting.

3. Research Recipe

- **Menu:** `Create > StackCraft > Special Recipes > Research Recipe`
- **Usage:** Used to unlock new recipes.
- **Logic:** Upon completion, it ignores the `ResultingCard`. Instead, it searches the database for a random **undiscovered** recipe and spawns a specific "Recipe Card" for the player to learn.

4. Travel Recipe

- **Menu:** `Create > StackCraft > Special Recipes > Travel Recipe`
- **Usage:** Used to move to new maps (e.g., Portal + Villager).
- **Logic:** Triggers a scene transition to one of the scenes defined in the `TargetScenes` list, carrying the stacked cards (travelers) to the new world.

5.3 Setting Up Quests

Quests in **StackCraft** serve as the tutorial and progression guide. They are defined as ScriptableObjects but, unlike Cards and Recipes, they must be explicitly assigned to the `QuestManager` in the scene to be active.

5.3.1 Defining a Quest

To create a new quest asset:

1. Right-click in the Project window.
2. Navigate to **Create > StackCraft > Quest**.

The Quest Inspector is divided into three logical sections:

1. **Info**
   - **ID:** Unique identifier (auto-generated).
   - **Title:** Short name displayed in the Quest UI.
   - **Description:** Detailed instructions for the player.

2. **Requirements** This determines *how* the quest is completed. You must select a `QuestType` and fill in the relevant target fields (`TargetCard`, `TargetRecipe`, or `TargetAmount`).

Quest Type	Logic & Requirement	Relevant Fields
Have	Checks if the player currently owns specific cards or currency. Updates dynamically based on stats.	`TargetCard` (or null for generic count), `TargetAmount`.
Obtain	Accumulative. Counts every time a specific card is created/spawned.	`TargetCard`, `TargetAmount`.
Discover	Completes when a specific recipe is unlocked.	`TargetRecipe`.
Defeat	Accumulative. Counts every time a specific mob card is killed.	`TargetCard`, `TargetAmount`.
Craft	Accumulative. Counts every time a specific card is the result of a craft.	`TargetCard`, `TargetAmount`.
Sell	Accumulative. Counts cards sold via the market. If `TargetCard` is null, counts *any* card sold.	`TargetCard` (optional), `TargetAmount`.
Buy	Accumulative. Counts Packs purchased.	`TargetCard` (assign Pack Definition here), `TargetAmount`.
Equip	Completes immediately when the specific card is equipped to a villager.	`TargetCard`.
Explore	Completes when an `ExplorationRecipe` finishes using the specified Area card.	`TargetCard` (Area).
Stats	Types like Food, Coins, Capacity check the global stat values directly.	`TargetAmount`.

3. **Flow (Chaining)** Quests are rarely standalone; they form a chain.
    - **Prerequisite Quests:** This quest will remain `Inactive` until *all* quests in this list are completed.
    - **Quests to Unlock:** (Optional/Redundant) While the system primarily checks prerequisites, you can list quests here for reference or future specific unlock logic.

5.3.2 Registering the Quest

Unlike Cards and Recipes which are loaded automatically from the `Resources` folder, Quests must be manually registered.

1. Select the **QuestManager** GameObject in your scene.
2. Locate the **Quest Groups** list in the Inspector.
3. Add your new Quest asset to the appropriate group (or create a new group).

5.4 Creating Encounters

Encounters are scheduled events that spawn cards onto the board automatically based on the current in-game day. This system is used for everything from daily resource deliveries to periodic enemy raids.

5.4.1 Defining an Encounter

To create a new encounter event:

1. Right-click in the Project window.
2. Navigate to **Create > StackCraft > Encounter**.
3. Place the asset inside `Resources/Encounters` (optional, but recommended for organization).

The Encounter Inspector is divided into three sections:

1. **Identity & Visuals**
    - **ID:** Unique identifier (auto-generated). Used to track if "One Time Only" events have already occurred.
    - **Notification Message:** Flavor text displayed to the player when the encounter triggers. If left empty, the spawn will happen silently.
2. **Spawn Settings**
    - **Card To Spawn:** The `CardDefinition` that will appear on the board.
    - **Count:** How many instances of that card should be created simultaneously.
    - **One Time Only:** If checked, this encounter will never fire again in the current save file once it has succeeded once.
3. **Conditions (The "When")** The `EncounterType` determines the scheduling logic:

Encounter Type	Usage	Inspector Field
Specific Day	One-off events (e.g., a tutorial gift on Day 1).	On Day
Recurring	Periodic events (e.g., a raid every 7 days).	Every X Days
Minimum Day	Random chance events starting after a certain point.	Starting From Day
Range	Events that only happen during a specific window.	From Day / Until Day

4. **Constraints (The "How Often")**
    - **Priority:** If multiple encounters are eligible on the same day, the manager processes them in order of priority (lower number = higher priority).
    - **Chance:** A value between 0 (0%) and 1 (100%) determining the probability of the event firing if the day condition is met.
    - **Max Cards On Board Limit:** A safety valve. The encounter will not fire if the number of cards already on the board exceeds this value, preventing performance lag or board clutter.

5.4.2 Registering Encounters

Like Quests, Encounters must be explicitly registered with the manager in your scene.

1. Select the **EncounterManager** GameObject.
2. Add your `EncounterDefinition` assets to the **All Encounters** list.

5.4.3 How Spawning Works

When an encounter triggers, the `EncounterManager` picks random coordinates within the board's valid bounds (respecting padding and top margins).

If the game is in **Friendly Mode**, any encounters set to spawn **Aggressive Mobs** are automatically skipped to ensure a peaceful experience.

5.5 Customizing Card & Pack Appearance

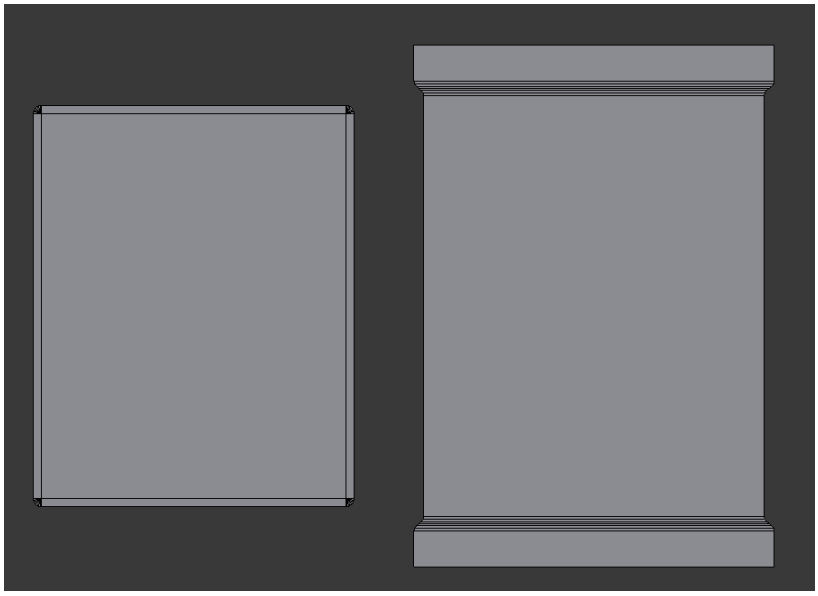
While the gameplay logic is handled by ScriptableObjects, the visual identity of **StackCraft** relies on a specific 3D setup and a specialized shader system. This section explains how to prepare assets and materials for your cards and card packs.

5.5.1 3D Models & Geometry

Both Cards and Packs are 3D meshes designed to feel like physical tabletop components.

- **Card**: A flat rectangular mesh with softly rounded corners, suitable for standard card presentations.

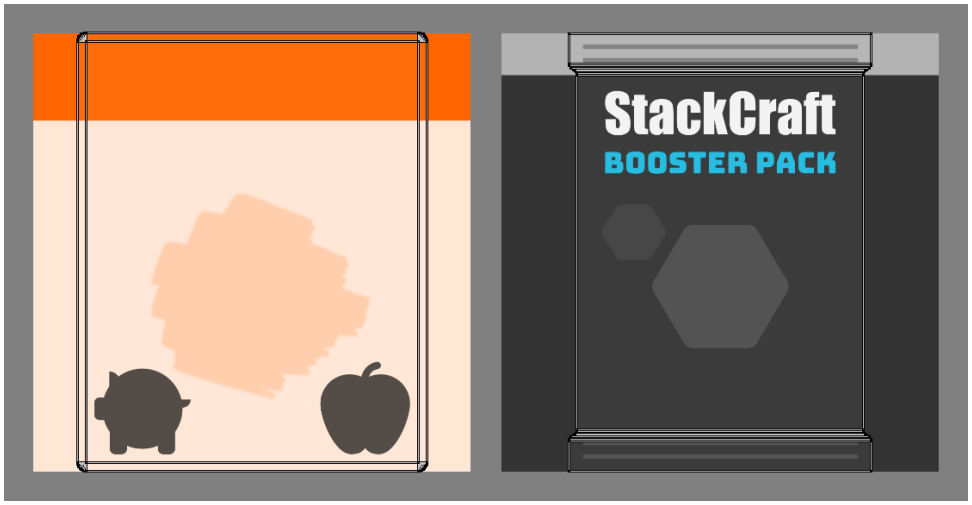
- **Pack**: A flat rectangular mesh representing a booster pack. Unlike cards, the top and bottom edges feature crimped folds, giving it a distinctive sealed-pack appearance.



5.5.2 Texture Guidelines & UV Mapping

To ensure high performance and cross-platform compatibility, all textures in **StackCraft** follow the **Power of Two (Po2)** rule.

- **Resolution:** Always use square textures with dimensions that are powers of two (e.g., 512x512, 1024x1024, or 2048x2048). This allows Unity to use efficient texture compression (like DXT or ASTC) and mipmapping.
- **UV Layout:** Because the card model is a vertical rectangle but the texture is a square, the UVs are centered.
    - The left and right margins of your square texture will not be visible on the card.
    - Design your frames within the central vertical strip of the square texture to avoid clipping.



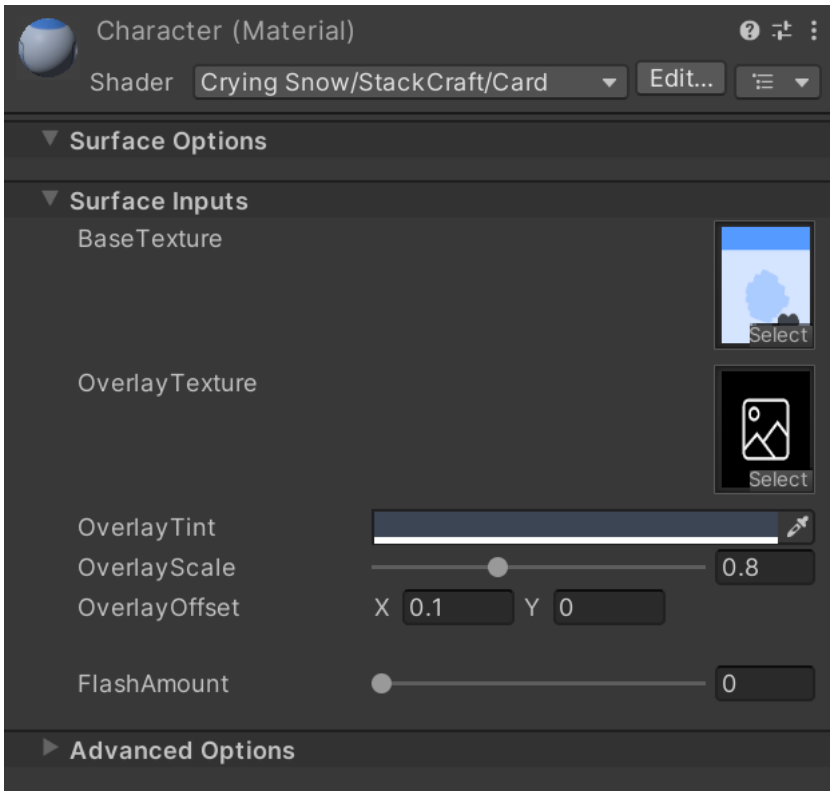
5.5.3 The Custom Shader System

StackCraft uses a custom multi-layered shader to render cards and packs efficiently. Instead of creating a separate texture for every single card in the game, the shader combines two distinct elements at runtime:

1. **Base Texture:** The underlying background of the card.
2. **The Artwork:** The specific illustration for the item or characater.

Assigning Textures:

- **Static Elements:** The **Base texture** is assigned directly in the Material asset.
- **Dynamic Elements:** The Artwork or **Overlay Texture** (defined in the `CardDefinition` as `artTexture`) is injected into the material by the `CardInstance` script at runtime when the card is spawned.



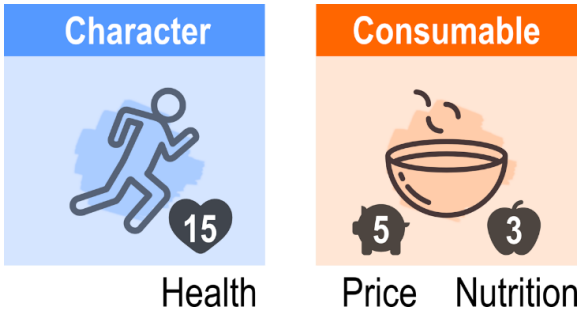
5.5.4 Category-Based Materials

Unlike Packs, Cards use different materials based on their `CardCategory` for two primary reasons:

1. Color Coding & Branding Each category (e.g., *Resource, Mob, Structure*) has a unique color palette. By using unique materials, you can change the "feel" of a card type globally, for example, making all "Aggressive Mob" cards have a red, aggressive frame and all "Resource" cards have a natural colored frame.

2. Stat Display & Layout Different categories require different data to be visible to the player:

- **Character:** The material and its associated UI overlay are configured to display Health.
- **Consumable:** May show a Price and Nutrition.



These stats are *embedded* into the visual presentation through the base texture and world-space UI (`TextMeshPro` component), ensuring that players can read critical information at a glance without opening a separate menu.

5.6 Customizing Board & Environment

The board is the primary stage of **StackCraft**. Unlike a static background, the board is a dynamic 3D entity that reacts to the player's progression. This section explains how the environment is constructed and how to adjust its visual parameters.

5.6.1 The Dynamic Board Model

The board is a specialized 3D model designed to maintain its visual integrity as the playable area expands or contracts.

1. **Geometry & Blendshapes:** The board is a flat plane with rounded corners. Rather than using standard Unity scaling (which would stretch and distort the corner radii), the board uses **Blendshapes**.
    a. **Preserving Edges:** Blendshapes allow the board to transition between its minimum and maximum dimensions while the rounded corners remain perfectly uniform.
    b. **Scaling Logic:** The board's size is controlled at runtime based on the number of `LimitBooster` cards currently on the board.

2. **Structure:** The model is split into two distinct meshes:
    a. **Header:** The top margin area where "Trade Zones" (Card Buyer, Pack Vendors) are located.
    b. **Body:** The main area where the player stacks and organizes cards.
3. **Materials:** Both the Header and Body use simple materials with exposed **Color** properties. You can customize the look of your game by simply changing these hex codes in the Material Inspector.

5.6.2 The Background & Backdrop

To provide a sense of depth and world-building, a background layer sits beneath the board.

- **Background Plane:** A simple 3D plane positioned slightly below the Board mesh.
- **Material:** It uses an **Unlit Material** to ensure it is unaffected by scene lighting, keeping the colors consistent and vibrant.
- **Seamless Patterns:** The texture applied here should be a **seamless (tileable) pattern**. Because the camera can move and the board can scale, a tileable texture ensures there are no visible "seams" or edges in the world background.

5.6.3 Lighting & Atmosphere

StackCraft relies on a clean, stylized look that is easy to read. This is achieved through a minimalist lighting setup:

- **Directional Light:** This is the primary and only light source in the scene. It provides the shadows and highlights for the 3D cards and packs. You can rotate this light to change the shadow direction or adjust the intensity to fit your art style.
- **Ambient Color:** Global lighting is further refined in the **Window > Rendering > Lighting Settings**.
    - By adjusting the **Ambient Color**, you can control the "fill" light of the scene.
    - *Tip:* Use a soft blue or warm tint in the ambient settings to give the shadows a more professional, stylized feel without adding extra light sources that could hurt performance.

5.7 Creating New Gameplay Scenes

In **StackCraft**, gameplay scenes function as distinct levels (e.g., Main, Island). Each scene acts as a container for its own set of rules, economy, and progression.

5.7.1 Organization

All scenes are located in the `Assets/StackCraft/Scenes` directory. While the Title scene handles game initialization, Gameplay scenes hold the actual board, managers, and world state.

5.7.2 Creating a New Level

The most efficient way to create a new gameplay scene is to duplicate an existing one (such as "Main") to ensure all required Manager GameObjects, environment, UI, and camera setups are preserved.

1. **Duplicate & Rename:** Select an existing gameplay scene in the Project window and press `Ctrl+D` (or `Cmd+D`). Rename it according to your level's theme.
2. **Visual Tweaks:** Follow the steps in **Section 5.6** to differentiate the atmosphere:
    - Change the **Board Materials** (Header and Body colors).
    - Swap the **Background Plane** texture for a new seamless pattern.
    - Adjust the **Directional Light** rotation or **Ambient Color** to match the new setting.

5.7.3 Configuring Local Managers

Once the environment is set, you must customize several core managers to define how this specific level plays.

1. **CardManager**
    - **Default Spawn Cards:** Define the starting state of the board. This is where you assign the "Starter Pack" or basic survival resources (e.g., a Villager and some Berry Bushes) that appear when the player first enters this scene.
2. **TradeManager** This defines the local economy of the level.
    - **Currency Type:** Assign the primary `CardDefinition` used for transactions. One scene might use Coins, while another uses Corals, Shells, or Gems.
    - **Offered Packs:** Define which "Pack Vendors" (Card Packs) are available for purchase in this level’s trade zone.
3. **QuestManager**
    - **Quest Groups:** Create new `QuestGroups` and unique list of `Quest` assets. You can create a entirely new tutorial chain or reuse existing quests to provide a sense of continuity.

4. **EncounterManager**
    - **All Encounters:** Add `EncounterDefinition` assets specific to this level. This allows you to have "Goblin Invasion" in one scene and "Raiding Pirate" in another.
5. **Other Managers** They can generally be left with their default settings.

5.7.4 Finalizing the Scene

Before the game can transition to your new level (via a `TravelRecipe` or the Title menu), you must register it with Unity:

1. Open File > Build Settings.
2. Drag your new scene from the Project window into the Scenes In Build list.
3. Ensure the scene name matches exactly the strings used in any `TravelRecipe` target lists.

5.8 Scene Travel & Linking

Traveling between gameplay scenes is handled through a specialized recipe system. This allows players to move to new regions while physically bringing a subset of their cards with them.

5.8.1 The Travel Recipe

To enable scene transitions, you must create a **Travel Recipe** asset.

- **Menu Path:** `Create > StackCraft > Special Recipes > Travel Recipe.`
- **Target Scenes:** This is a list of scene names (matching your Build Settings). The system treats this list as a **circular loop**. When the recipe executes, the `GameDirector` finds the current scene in this list and moves the player to the *next* scene in the sequence.

5.8.2 How Travel Works

When a stack matches a Travel Recipe (e.g., a "Portal" card with a "Villager" stacked on top):

1. **Selection of Travelers:** Every card currently in the stack at the moment the progress bar finishes is marked as a **Traveler**.
2. **State Persistence:** The `GameDirector` captures the full data of these specific cards (health, durability, stats) and stores them in a temporary `incomingTravelers` buffer.
3. **Global Saving:** Before the scene changes, the game performs an automatic **SaveGame()** to ensure the state of the *current* scene is preserved for when the player eventually returns.

4. **The Sequence:**
    - The `TimeManager` pauses the game simulation.
    - The `ScreenFader` triggers a fade-to-black.
    - The new scene is loaded asynchronously.
    - Once loaded, the game fades back in and resumes time.

5.8.3 Spawning on Arrival

Once the new scene is initialized, the `GameDirector` calls `SpawnTravelers()`.

- All cards from the previous scene's stack are instantiated at random positions on the new board.
- The `CardManager` uses `RestoreTraveler()` to ensure the cards retain their exact state (e.g., if a Villager was hungry or a tool was damaged, those values remain unchanged).

5.8.4 Best Practices for Linking

- **Safety Requirements:** Since the travel list is circular, ensure every scene in a "world loop" contains a Travel Recipe that points back to the others.
- **Ingredient Consumption:** By default, the "Vehicle" card (like a Boat or Portal) is usually consumed or "Kept" based on your `IngredientConsumption` settings in the recipe asset. If you want the boat to travel *with* the player, ensure it is part of the stack and not destroyed by the recipe rules.
- **Scene Names:** Double-check that the strings in the `targetScenes` list exactly match the names in `Assets/StackCraft/Scenes`.

6. Third-Party Assets

StackCraft utilizes several high-quality third-party libraries and assets to enhance the gameplay experience and development workflow. This section lists the attributions and licenses for the external content included in this project.

6.1 Coding & Animation Tools

- **DOTween (v1.2.765)**
    - **Usage:** Used for card animations, UI transitions, and smooth movement interpolations throughout the game.
    - **License:** MIT License.
    - **Author:** Daniele Giardini.

6.2 Visual Assets & Icons

- **SVG Repo Icons**
    - **Usage:** Much of the card artwork and iconography in this package is based on icons sourced from SVG Repo. These icons have been modified, recolored, or combined to fit the **StackCraft** aesthetic.
    - **License:** Creative Commons Zero (CC0) Public Domain.
    - **URL:** https://www.svgrepo.com/

6.3 Audio & Music

- **Background Music (BGM)**
    - **Included Track:** *Medieval: King's Feast* by RandomMind.
    - **License:** Creative Commons Zero (CC0) Public Domain.
    - **Usage:** Provides the ambient atmosphere for the Title and Gameplay scenes.

Final Documentation Note

This concludes the **StackCraft** Documentation and User Guide. By utilizing the ScriptableObject-driven systems (Cards, Recipes, Quests, and Encounters) alongside the dynamic Board and Travel systems, you can expand this project into a vast, multi-region crafting adventure.

Happy Crafting!
Crying Snow